Wanderlust — Trip Discovery App (Bulletproofed Spec v2)

What It Is

A lightweight “help me pick a destination” tool. User inputs where they are, how far they want to go, how they’re traveling, and what vibe they want — app returns 5–8 matching destinations. Not a booking engine. A discovery engine.

Spec Audit — Issues Found & Resolved

Issue	Original	Resolution
Drive cap conflict	Page 2 says ≤600mi, filter logic says ≤800mi	**800mi** — 600 cuts legit trips (Buffalo→Nashville is ~590mi). Slider already lets users self-select.
Fly floor conflict	Page 2 says >300mi, filter logic says ≥200mi	**200mi** — prevents a dead zone between drive cap and fly floor
No error state for unresolved city	Not mentioned	Added: warm empty state with suggestions
No sort toggle UI placement	“Sort by” mentioned, no UI spec	Added: pill toggle row above results grid
No tiebreaker in scoring	Score formula could produce ties	Added: secondary sort by alphabetical name
Dark mode toggle requested	Not in original MVP	Added: CSS class strategy, NO localStorage (respects privacy rules), uses `prefers-color-scheme` detection with manual toggle
Share/copy result link	Not in original	Added: “Copy link” button generates shareable URL with search params encoded in URL hash (no server, no tracking)

Weather hint on cards	Not in original	Added: seasonal hint text derived from `bestSeasons[]` + current month (still zero API calls)
`next-themes` uses localStorage	Conflicts with zero-localStorage rule	Resolved: custom dark mode toggle using CSS class + React state only. No `next-themes` dependency. No localStorage.
City dataset hallucination risk	120 cities in one array	Resolved: regional batch architecture — 10 regions × 12 cities, explicit coords provided per batch
Keywords input ambiguity	"Coming Soon" badge but still typeable	Clarified: `tabIndex={-1}` so it's visible but not focusable, `pointer-events-none` on the input
Google Fonts privacy	Spec says "only external requests are Google Fonts"	Resolved: `next/font/google` self-hosts fonts at build time — **zero runtime requests to Google**. True zero external calls.
No accessibility spec	Not mentioned	Added: ARIA labels, keyboard navigation, focus rings, color contrast requirements
No loading/transition state	What happens between form submit and results?	Added: brief skeleton shimmer on cards during filter calc (even though it's instant — gives visual feedback)

MVP Scope (What Ships)

User Inputs

- **Home city** — text input with autocomplete dropdown (filtered from cityLookup map)
- **Max distance** — range slider (50mi–3000mi) with dynamic label: "Day Trip" (≤150mi) / "Weekend Drive" (≤400mi) / "Road Trip" (≤800mi) / "Cross Country" (≤1500mi) / "Fly Me Out" (>1500mi)
- **Mode** — two-button toggle: 🚗 Drive | ✈️ Fly
- **Trip length** — four pill buttons: 1 Day, Weekend, 3–5 Days, Week+
- **Vibe tags** — multi-select chips (10 vibes)

- **Keywords** — text field with “Coming Soon ✨” badge, visible but disabled (`pointer-events-none` , `tabIndex={-1}`)
- **Dark mode toggle** — sun/moon icon button in top-right corner

Output

- Card grid of 5–8 matching destinations
- Each card: city name, state, distance badge (“342 mi”), 2–3 sentence blurb, matching vibe pills, best seasons text, seasonal weather hint
- Sort toggle row: Best Match (default) | Closest | Furthest
- Share button per card (copies a pre-formatted text snippet to clipboard)
- Empty state with compass emoji and helpful message
- Error state for unresolved home city with “Did you mean…?” suggestions

Data Strategy

- Static JSON dataset of **120 US cities** organized in **10 regional batches** (12 per region)
- Haversine distance calc client-side (miles, Earth radius = 3958.8mi)
- City lookup map of ~200 US cities (all 120 + ~80 common origin cities)
- **Zero API calls** — everything runs client-side
- Filter logic: mode filter → distance filter → vibe scoring → sort → top 8

Tech Stack

- Next.js 14 (App Router)
- TypeScript (strict, no `any`)
- Tailwind CSS (with `darkMode: 'class'`)
- `next/font/google` for Fraunces + DM Sans (self-hosted at build time)
- Vercel deploy (zero config, zero env vars)
- **No dependencies beyond**: next, react, react-dom, tailwindcss, postcss, autoprefixer, @types/react, @types/node

---

Privacy & Attribution (Hard Rules)

1. Zero tracking. No analytics, no cookies, no localStorage, no sessionStorage, no external API calls at runtime.
2. `next/font/google` self-hosts fonts — zero runtime Google requests.
3. Footer (low opacity, small text):
   - Line 1: "100% client-side. No data stored. No tracking. Your info never leaves your browser."
   - Line 2: "@ReelDad on X" linked to `https://x.com/ReelDad`
4. No personal info anywhere in codebase beyond the X handle above. No real names, company names, project names, IP addresses, API keys, secrets, version numbers.

Architecture

```
/app
  /page.tsx              ← Single page: form + results
  /layout.tsx            ← Root layout, fonts, meta, dark mode wrapper
  /globals.css           ← Tailwind directives + CSS custom properties + dark mod
/components
  /SearchForm.tsx        ← All inputs as single controlled component
  /ResultsGrid.tsx       ← Grid of cards with stagger animation + sort controls
  /DestinationCard.tsx   ← Individual result card with share button
  /VibeChip.tsx          ← Selectable tag chip
  /DarkModeToggle.tsx    ← Sun/moon toggle, CSS class strategy, no localStorage
  /CityAutocomplete.tsx  ← Filtered dropdown for home city input
/data
  /cities.ts             ← Static dataset (120 entries in 10 regional batches)
  /vibes.ts              ← Vibe definitions with emoji icons
/lib
  /distance.ts           ← Haversine formula (miles)
  /filter.ts             ← filterDestinations() + scoring
  /cityLookup.ts         ← City name → lat/lng resolver with fuzzy matching
  /seasons.ts            ← getCurrentSeason() + getWeatherHint()
  /shareUtils.ts         ← Copy-to-clipboard helper for share feature
/api
  /suggest/route.ts      ← Future AI endpoint stub (returns 501)
/tailwind.config.ts      ← Extended theme with custom colors + fonts + darkMode:
/package.json
/tsconfig.json
```

```
/next.config.js
```

Data Schemas

City

```
interface City {
  name: string;
  state: string;           // Full state name
  lat: number;
  lng: number;
  vibes: VibeTag[];
  populationTier: 'small' | 'medium' | 'large' | 'metro';
  bestSeasons: Season[];
  blurb: string;           // 2-3 sentences, why visit. Specific and evocative.
  events?: string[];       // Future use — always empty array for now
}

type VibeTag = 'beach' | 'mountains' | 'city' | 'outdoors' | 'nightlife' | 'histo
type Season = 'spring' | 'summer' | 'fall' | 'winter';
```

Search Params

```
interface SearchParams {
  homeCity: string;
  maxDistance: number;      // miles, 50-3000
  mode: 'drive' | 'fly';
  tripLength: 'day' | 'weekend' | '3-5days' | 'week';
  vibes: VibeTag[];
  keywords: string;        // Captured but NOT filtered on (future use)
}
```

Scored City (internal)

```
interface ScoredCity extends City {
  distance: number;        // miles from home city
  score: number;           // computed match score
  weatherHint: string;     // e.g. "Great time to visit!" or "Peak season"
}
```

Vibes Definition

```
const VIBES = [
  { id: 'beach',     label: 'Beach',              emoji: '⛱️' },
  { id: 'mountains', label: 'Mountains',           emoji: '⛰️' },
  { id: 'city',      label: 'City Life',           emoji: '🏙️' },
  { id: 'outdoors',  label: 'Outdoors',            emoji: '🌲' },
  { id: 'nightlife', label: 'Nightlife',           emoji: '🌃' },
  { id: 'history',   label: 'History',             emoji: '🏛️' },
  { id: 'food',      label: 'Food Scene',          emoji: '🍽️' },
  { id: 'offbeat',   label: 'Off the Beaten Path', emoji: '🗺️' },
  { id: 'family',    label: 'Family Friendly',     emoji: '👨‍👩‍👧‍👦' },
  { id: 'budget',    label: 'Budget Friendly',     emoji: '💰' },
] as const;
```

Filter Logic (`lib/filter.ts`)

```
function filterDestinations(cities: City[], params: SearchParams): ScoredCity[]
```

Step-by-step:

1. Resolve user's home city to `{lat, lng}` via `cityLookup`. If unresolved, return empty array (UI shows error state).
2. Calculate Haversine distance (miles) from home to every city.
3. **Distance filter**: `distance ≤ params.maxDistance`
4. **Mode filter**:
   - If `mode === 'drive'` : only cities where `distance ≤ 800`
   - If `mode === 'fly'` : only cities where `distance ≥ 200`
5. **Scoring** (per city):
   - `+10` per matching vibe tag (intersection of user's vibes and city's vibes)
   - `+5` if current season is in city's `bestSeasons[]`
   - `-1` per 100mi distance (Math.floor(distance / 100)) — slight closeness preference
   - Minimum score floor of 0 (never negative)

6. **Filter out zero-score** cities if user selected at least one vibe (prevents irrelevant results)
7. **Sort**: by score descending, then alphabetical by city name (tiebreaker)
8. **Return top 8**

City Lookup (`lib/cityLookup.ts`)

- `Map<string, {lat: number, lng: number}>` with ~200 entries
- Key format: `"city, state"` lowercase (e.g., `"buffalo, new york"`)
- **Fuzzy lookup function** `resolveCity(input: string): {lat: number, lng: number} | null` :
  1. Trim, lowercase
  2. Try exact match
  3. Try with common abbreviations: `"ny"→"new york"` , `"ca"→"california"` , etc.
  4. Try matching city name only (first match wins)
  5. Handle aliases: `"nyc"→"new york, new york"` , `"la"→"los angeles, california"` , `"sf"→"san francisco, california"` , `"dc"→"washington, district of columbia"` , `"chi"→"chicago, illinois"` , `"philly"→"philadelphia, pennsylvania"` , `"vegas"→"las vegas, nevada"` , `"nola"→"new orleans, louisiana"`
  6. Return `null` if no match (UI handles the error)

Haversine Formula (`lib/distance.ts`)

```
function haversineDistance(
  lat1: number, lng1: number,
  lat2: number, lng2: number
): number  // returns miles
```

- Earth radius: **3958.8 miles**
- Convert degrees to radians
- Standard Haversine: `a = sin²(Δlat/2) + cos(lat1) * cos(lat2) * sin²(Δlng/2)`

- `c = 2 * atan2(√a, √(1-a))`
- `distance = R * c`
- Return as **miles** (not km)

Season & Weather Hint (`lib/seasons.ts`)

```
function getCurrentSeason(): Season {
  const month = new Date().getMonth(); // 0-11
  if (month >= 2 && month <= 4) return 'spring';
  if (month >= 5 && month <= 7) return 'summer';
  if (month >= 8 && month <= 10) return 'fall';
  return 'winter';
}

function getWeatherHint(city: City): string {
  const current = getCurrentSeason();
  if (city.bestSeasons.includes(current)) {
    return '☀️ Great time to visit!';
  }
  const nextGood = city.bestSeasons[0]; // first listed = primary
  return `Best in ${nextGood}`;
}
```

Dark Mode Implementation

Constraints: Zero localStorage, zero cookies, zero external dependencies.

Approach:

1. `tailwind.config.ts` : `darkMode: 'class'`
2. `DarkModeToggle.tsx` : Client component with React state only
3. On mount, detect `window.matchMedia('(prefers-color-scheme: dark)')` and set initial state
4. Toggle adds/removes `'dark'` class on `document.documentElement`
5. All color vars have light + dark variants defined in `globals.css`
6. State is ephemeral — refreshing resets to OS preference (this is intentional, respects the

zero-persistence privacy rule)

Dark mode color palette (extends the warm palette):

- `--sand-dark: #1A1410` (dark bg)
- `--cream-dark: #2A2420` (dark card bg)
- `--charcoal-dark: #E8E0D8` (dark text — warm off-white)
- `--terracotta` and `--coral` stay the same (accent colors work on both)
- `--forest-dark: #4A8A30` (brighter green for dark bg contrast)
- `--sage-dark: #6B8A60`

Share Feature (`lib/shareUtils.ts`)

How it works:

- Each `DestinationCard` has a small share icon button
- On click, copies a text snippet to clipboard:

  ```
  🧭 Wanderlust pick: Austin, Texas (342 mi) — "Keep Austin weird and your stoma
  ```

- Uses `navigator.clipboard.writeText()` with fallback to legacy `document.execCommand('copy')`
- Brief "Copied!" tooltip on success (2s fade)
- **No URL sharing** (would require encoding state, adds complexity for MVP). Just a clean text snippet.

UI / Design Spec

Color System (CSS Custom Properties)

Light mode:

- `--sand: #F5E6D3` (page bg)
- `--terracotta: #C75B39` (primary accent)

- `--forest`: `#2D5016` (secondary accent)
- `--cream`: `#FFF8F0` (card bg)
- `--charcoal`: `#2C2C2C` (body text)
- `--coral`: `#FF6B4A` (CTA / highlight)
- `--sage`: `#87A878` (subtle accent)

Dark mode (applied when `.dark` class on `<html>`):

- `--sand`: `#1A1410`
- `--cream`: `#2A2420`
- `--charcoal`: `#E8E0D8`
- `--forest`: `#4A8A30`
- `--sage`: `#6B8A60`
- `--terracotta` and `--coral`: unchanged

Typography

- **Display**: Fraunces (Google Fonts via `next/font/google`, weights 400/700/900) — h1, h2, card city names
- **Body**: DM Sans (Google Fonts via `next/font/google`, weights 400/500/700) — body, labels, inputs, card text
- Both loaded via `next/font/google` with `display: 'swap'` and CSS `variable` mode, applied to `<body>` className

Layout

- Max-width `1200px`, centered with auto margins
- **Hero**: Large "Wanderlust" in Fraunces 900, tagline "Where should you go next?" underneath. Subtle CSS grain/noise texture bg (inline SVG data URI, not an external file). Dark mode toggle floats top-right.
- **Form section**: Card on `--cream` bg, generous padding. Inputs stacked on mobile, 2-column grid on desktop (≥768px).
- **Distance slider**: Custom-styled range input. Shows current value in miles + text label that updates dynamically.

- **Mode toggle**: Two adjacent buttons, selected state has `--terracotta` bg. Drive shows 🚗, Fly shows ✈️.
- **Trip length**: Four pill buttons in a row, selected = `--forest` bg with white text.
- **Vibes**: Flex-wrap row of `VibeChip` components. Unselected = outline on `--sage`. Selected = `--terracotta` bg, white text, subtle scale bounce (`transform: scale(1.05)` on select, 150ms).
- **Keywords**: Text input with "Coming Soon ✨" badge overlaid top-right, input has `opacity-60`, `pointer-events-none`, `tabIndex={-1}`.
- **Discover button**: Full-width, `--coral` bg, white text, Fraunces 700, large (text-xl), `rounded-lg`. Subtle idle pulse animation (CSS `@keyframes pulse`).
- **Sort controls**: Row of three text pills above results. Active has underline in `--terracotta`.

Results Section

- Section heading: "Your Next Adventure" (Fraunces 700)
- 2-column grid on desktop, 1-column on mobile
- **Cards**: `--cream` bg, `rounded-xl`, subtle `box-shadow`. Contain:
  - City name (Fraunces 700, `text-xl`)
  - State + distance badge (DM Sans, `--sage` colored badge with pill shape)
  - Weather hint (small text, color-coded: green for "Great time!", muted for "Best in...")
  - Blurb (DM Sans 400, `text-sm`, `--charcoal`)
  - Matching vibe tags as small pills (emoji + label, muted bg)
  - Best seasons as subtle text at bottom
  - Share button (small icon, top-right corner of card)
- **Stagger animation**: Cards slide up + fade in. Use CSS `animation` with `animation-delay` incremented by `100ms` per card index (set via inline `style={{ animationDelay }}` — this is the ONE exception to "no inline styles"). Keyframes: `@keyframes slideUp { from { opacity: 0; transform: translateY(20px); } to { opacity: 1; transform: translateY(0); } }`

Empty & Error States

- **Empty** (no search yet): “Tell us what you’re looking for and we’ll find your next adventure 🧭” — centered, Fraunces italic
- **No results**: “No destinations matched your criteria. Try adjusting your distance or vibes!” — with a subtle illustration placeholder (emoji compass)
- **City not found**: “We couldn’t find that city. Try entering a city and state, like ‘Austin, Texas’” — shows below the home city input in `--coral` text

Responsive Breakpoints

- Mobile-first design
- `sm` (640px): minor spacing adjustments
- `md` (768px): form goes 2-column, results stay 1-column
- `lg` (1024px): results go 2-column
- Slider always full-width

Animations (CSS only, no JS animation libraries)

- Page load: title fades in (`@keyframes fadeIn`), form slides up
- Vibe chips: `transition: transform 150ms ease` on select
- Results: stagger slide-up (CSS `animation-delay` , NOT JavaScript)
- Discover button: subtle `@keyframes pulse` idle animation (opacity 0.8→1.0→0.8, 3s loop)
- Dark mode transition: `transition: background-color 300ms ease, color 300ms ease` on `body`

Accessibility

- All interactive elements have visible focus rings (`outline-2 outline-offset-2 outline-coral`)
- Vibe chips: `role="checkbox"` , `aria-checked`
- Mode toggle: `role="radiogroup"` with `role="radio"` per button
- Slider: `aria-label="Maximum travel distance in miles"` , `aria-valuetext` with label
- Dark mode toggle: `aria-label="Toggle dark mode"`

- Color contrast: all text meets WCAG AA (4.5:1 minimum)

City Dataset — Regional Batch Architecture

Build ALL 120 cities with **real, accurate lat/lng coordinates** (verify against Google Maps). Organized in 10 regional batches of 12 cities each. Every city must have a specific, evocative blurb (not generic tourism copy).

Regions (12 cities each):

1. **Northeast** — NYC, Boston, Philadelphia, Portland ME, Providence, Burlington VT, Cape May NJ, Bar Harbor ME, Newport RI, Saratoga Springs NY, Ithaca NY, Mystic CT
2. **Southeast** — Miami, Charleston SC, Savannah GA, Asheville NC, Nashville TN, Key West FL, St. Augustine FL, Chattanooga TN, Wilmington NC, Beaufort SC, Hot Springs AR, Oxford MS
3. **Mid-Atlantic** — Washington DC, Baltimore, Richmond VA, Virginia Beach, Charlottesville VA, Annapolis MD, Gettysburg PA, Shenandoah VA, Pittsburgh, Lancaster PA, Rehoboth Beach DE, Harrisburg PA
4. **Midwest** — Chicago, Minneapolis, Detroit, Milwaukee, Indianapolis, Cleveland, Columbus OH, Kansas City MO, Madison WI, Ann Arbor MI, Duluth MN, Traverse City MI
5. **South Central** — Austin, San Antonio, New Orleans, Memphis TN, Little Rock AR, Galveston TX, Fredericksburg TX, Natchez MS, Eureka Springs AR, Branson MO, Shreveport LA, Biloxi MS
6. **Mountain West** — Denver, Salt Lake City, Boise, Missoula MT, Jackson WY, Park City UT, Telluride CO, Whitefish MT, Sun Valley ID, Moab UT, Durango CO, Bozeman MT
7. **Southwest** — Phoenix, Santa Fe NM, Tucson, Sedona AZ, Albuquerque, Las Vegas, Flagstaff AZ, Taos NM, Scottsdale AZ, Page AZ, Marfa TX, Roswell NM
8. **West Coast** — Los Angeles, San Francisco, San Diego, Portland OR, Seattle, Santa Barbara, Monterey CA, Bend OR, Olympia WA, Carmel CA, Laguna Beach CA, Half Moon Bay CA
9. **Pacific & Islands** — Honolulu, Maui HI, Kauai HI, Juneau AK, Anchorage, Fairbanks AK, Sitka AK, Ketchikan AK, Hilo HI, Kona HI, Kodiak AK, Waimea HI
10. **Plains & Frontier** — Rapid City SD, Bismarck ND, Sioux Falls SD, Fargo ND, Omaha NE, Wichita KS, Oklahoma City, Tulsa, Des Moines IA, Lincoln NE, Cheyenne WY, Billings MT

Each batch instruction to Claude Code: “For each city, provide the EXACT lat/lng to 4

decimal places. Cross-reference against known coordinates. If unsure, use the city center coordinates. Every blurb must be 2-3 sentences describing what makes the city special — specific landmarks, food, culture, or natural features. No generic 'vibrant city with lots to do' copy."

Future-Ready Stubs

1. **AI Search endpoint** — `/api/suggest/route.ts` exists, returns `NextResponse.json({ message: 'AI suggestions coming soon — for now, enjoy our curated picks!' }, { status: 501 })`. Will eventually accept SearchParams + keywords, hit Claude API with web search for events/deals.
2. **Keywords input** — wired to state, captured in SearchParams, but not used in filter logic. When AI endpoint goes live, keywords get sent there.
3. **Events field** — City schema has optional `events?: string[]`. Not populated. When AI endpoint goes live, can be populated dynamically.

Build Rules (Non-Negotiable)

1. Every file must be COMPLETE. No `// add more cities here` — write all 120.
2. Use `'use client'` directive on every component that uses hooks, event handlers, or browser APIs.
3. `cityLookup` must handle: "NYC", "New York", "new york, ny", "new york, new york", "New York, NY" — all resolve correctly.
4. Haversine formula uses **miles** (radius = 3958.8), NOT kilometers.
5. API stub route must exist and return proper JSON with 501 status.
6. All TypeScript — **zero `any` types**. Define every interface and type explicitly.
7. Tailwind classes only. No inline styles EXCEPT: CSS custom property references and `animationDelay` for stagger effect.
8. Google Fonts loaded via `next/font/google` (NOT `<link>` tags). Self-hosted at build time.
9. Must deploy on Vercel with zero config. No environment variables needed.
10. Footer: exactly as specified in Privacy section. Nothing else.

11. **Zero localStorage, zero sessionStorage, zero cookies, zero analytics, zero tracking pixels, zero external API calls at runtime.** The ONLY "external" resources are Google Fonts which `next/font` downloads at build time.
12. Dark mode toggle must work WITHOUT `next-themes`, WITHOUT localStorage. Pure React state + CSS class toggle + OS preference detection on mount.
13. Keyboard navigable. Focus rings visible. ARIA attributes on custom controls.
14. No `console.log` statements in production code (remove all debug logging).
15. All city coordinates must be real and accurate to 4 decimal places.

Timeline (Adjusted for Added Features)

Block	Time	Task
1	1.5h	Scaffold Next.js, Tailwind, types, data files
2	2h	Build city dataset (120 cities, 10 regional batches)
3	2h	Core UI: form + results + cards + dark mode
4	1.5h	Filter logic, distance calc, scoring, state mgmt
5	1h	Share feature, weather hints, city autocomplete
6	1h	Polish: animations, responsive, error states, a11y
7	0.5h	Deploy to Vercel, smoke test

ONE-SHOT CLAUDE CODE PROMPT

Everything between the `---PROMPT START---` and `---PROMPT END---` markers below is the optimized prompt to paste into Claude Code.

—PROMPT START—

Build a complete, production-ready Next.js 14 (App Router) application called "Wanderlust" — a trip destination discovery tool. TypeScript + Tailwind CSS. Ship-ready for Vercel with

zero config.

CRITICAL INSTRUCTIONS:

- Every single file must be COMPLETE with zero placeholders. No “// add more” comments.
- Build all 120 cities with real, verified lat/lng coordinates (4 decimal places).
- Zero `any` types. Define every interface and type.
- Test mentally that your Haversine formula returns MILES not km (radius = 3958.8 mi).
- No localStorage, no sessionStorage, no cookies, no analytics, no tracking, no external API calls at runtime.
- Before writing each file, state what you’re about to build and why. After finishing each major section (data, components, lib, etc.), do a quick self-audit for bugs.

What It Does

User enters home city, max travel distance, travel mode (drive/fly), trip length, and selects vibe tags. App shows 5-8 matching US city destinations as cards. Pure client-side filtering against a static dataset. No booking, no external APIs.

Added features beyond basic MVP:

- Dark mode toggle (CSS class strategy + React state, NO next-themes, NO localStorage)
- Share button on each card (copies text snippet to clipboard)
- Weather hint on cards (derived from bestSeasons[] + current month, zero API calls)
- City autocomplete dropdown on home city input
- Sort controls: Best Match | Closest | Furthest

File Structure

```
/app/page.tsx              — Main page with SearchForm and ResultsGrid
/app/layout.tsx            — Root layout with next/font/google (Fraunces + DM San
/app/globals.css           — Tailwind directives + CSS custom properties for ligh
/components/SearchForm.tsx — All form inputs as single controlled component (home
/components/ResultsGrid.tsx — Sort controls + grid of DestinationCards with stagg
/components/DestinationCard.tsx — Individual result card with share button and we
/components/VibeChip.tsx    — Selectable tag chip with emoji, scale animation on
/components/DarkModeToggle.tsx — Sun/moon toggle button. On mount: detect prefers
```

```
/components/CityAutocomplete.tsx — Filtered dropdown showing matching cities from
/data/cities.ts            — 120 US cities in 10 regional arrays, merged into sin
/data/vibes.ts             — Vibe definitions with emoji icons
/lib/distance.ts           — Haversine formula returning MILES (Earth radius = 39
/lib/filter.ts             — filterDestinations() with scoring algorithm
/lib/cityLookup.ts         — Map<string, {lat, lng}> of ~200 cities + fuzzy resol
/lib/seasons.ts            — getCurrentSeason() and getWeatherHint(city)
/lib/shareUtils.ts         — copyToClipboard(text) helper with fallback
/app/api/suggest/route.ts  — Stub: returns NextResponse.json({ message: 'AI sugge
/tailwind.config.ts        — darkMode: 'class', extended theme with color vars +
/package.json              — deps: next, react, react-dom, tailwindcss, postcss,
/tsconfig.json             — Standard Next.js TS config
/next.config.js            — Standard Next.js config (empty/minimal)
```

Type Definitions

```
// In data/cities.ts or a shared types file
type VibeTag = 'beach' | 'mountains' | 'city' | 'outdoors' | 'nightlife' | 'histo
type Season = 'spring' | 'summer' | 'fall' | 'winter';

interface City {
  name: string;
  state: string;
  lat: number;
  lng: number;
  vibes: VibeTag[];
  populationTier: 'small' | 'medium' | 'large' | 'metro';
  bestSeasons: Season[];
  blurb: string;
  events?: string[];
}

interface SearchParams {
  homeCity: string;
  maxDistance: number;
  mode: 'drive' | 'fly';
  tripLength: 'day' | 'weekend' | '3-5days' | 'week';
  vibes: VibeTag[];
  keywords: string;
}

interface ScoredCity extends City {
  distance: number;
  score: number;
  weatherHint: string;
```

```
}

type SortMode = 'best' | 'closest' | 'furthest';
```

Vibes Data

```
const VIBES = [
  { id: 'beach', label: 'Beach', emoji: '⛱️' },
  { id: 'mountains', label: 'Mountains', emoji: '⛰️' },
  { id: 'city', label: 'City Life', emoji: '🏙️' },
  { id: 'outdoors', label: 'Outdoors', emoji: '🌲' },
  { id: 'nightlife', label: 'Nightlife', emoji: '🌃' },
  { id: 'history', label: 'History', emoji: '🏛️' },
  { id: 'food', label: 'Food Scene', emoji: '🍽️' },
  { id: 'offbeat', label: 'Off the Beaten Path', emoji: '🗺️' },
  { id: 'family', label: 'Family Friendly', emoji: '👨‍👩‍👧‍👦' },
  { id: 'budget', label: 'Budget Friendly', emoji: '💰' },
] as const;
```

City Dataset (120 Cities — 10 Regional Batches)

Build 10 arrays, 12 cities each. Merge into single `CITIES` export.

CRITICAL: Every lat/lng MUST be accurate to the real city center (4 decimal places). Every blurb must be 2-3 specific, evocative sentences — never generic filler.

Region 1: Northeast (12 cities) New York City NY, Boston MA, Philadelphia PA, Portland ME, Providence RI, Burlington VT, Cape May NJ, Bar Harbor ME, Newport RI, Saratoga Springs NY, Ithaca NY, Mystic CT

Region 2: Southeast (12 cities) Miami FL, Charleston SC, Savannah GA, Asheville NC, Nashville TN, Key West FL, St. Augustine FL, Chattanooga TN, Wilmington NC, Beaufort SC, Hot Springs AR, Oxford MS

Region 3: Mid-Atlantic (12 cities) Washington DC, Baltimore MD, Richmond VA, Virginia Beach VA, Charlottesville VA, Annapolis MD, Gettysburg PA, Shenandoah VA (Luray as proxy), Pittsburgh PA, Lancaster PA, Rehoboth Beach DE, Harrisburg PA

Region 4: Midwest (12 cities) Chicago IL, Minneapolis MN, Detroit MI, Milwaukee WI, Indianapolis IN, Cleveland OH, Columbus OH, Kansas City MO, Madison WI, Ann Arbor MI, Duluth MN, Traverse City MI

Region 5: South Central (12 cities) Austin TX, San Antonio TX, New Orleans LA, Memphis

TN, Little Rock AR, Galveston TX, Fredericksburg TX, Natchez MS, Eureka Springs AR, Branson MO, Shreveport LA, Biloxi MS

Region 6: Mountain West (12 cities) Denver CO, Salt Lake City UT, Boise ID, Missoula MT, Jackson WY, Park City UT, Telluride CO, Whitefish MT, Sun Valley ID, Moab UT, Durango CO, Bozeman MT

Region 7: Southwest (12 cities) Phoenix AZ, Santa Fe NM, Tucson AZ, Sedona AZ, Albuquerque NM, Las Vegas NV, Flagstaff AZ, Taos NM, Scottsdale AZ, Page AZ, Marfa TX, Roswell NM

Region 8: West Coast (12 cities) Los Angeles CA, San Francisco CA, San Diego CA, Portland OR, Seattle WA, Santa Barbara CA, Monterey CA, Bend OR, Olympia WA, Carmel-by-the-Sea CA, Laguna Beach CA, Half Moon Bay CA

Region 9: Pacific & Islands (12 cities) Honolulu HI, Kahului (Maui) HI, Lihue (Kauai) HI, Juneau AK, Anchorage AK, Fairbanks AK, Sitka AK, Ketchikan AK, Hilo HI, Kailua-Kona HI, Kodiak AK, Waimea (Kamuela) HI

Region 10: Plains & Frontier (12 cities) Rapid City SD, Bismarck ND, Sioux Falls SD, Fargo ND, Omaha NE, Wichita KS, Oklahoma City OK, Tulsa OK, Des Moines IA, Lincoln NE, Cheyenne WY, Billings MT

City Lookup (~200 cities)

Build a Map<string, {lat: number, lng: number}> containing:

- All 120 cities from the dataset
- Plus ~80 additional common origin cities not in the dataset (e.g., Buffalo NY, Rochester NY, Syracuse NY, Albany NY, Hartford CT, Stamford CT, Newark NJ, Jersey City NJ, Tampa FL, Orlando FL, Jacksonville FL, Atlanta GA, Charlotte NC, Raleigh NC, Louisville KY, Cincinnati OH, St. Louis MO, Houston TX, Dallas TX, Fort Worth TX, El Paso TX, Colorado Springs CO, Sacramento CA, Fresno CA, Bakersfield CA, Oakland CA, San Jose CA, Reno NV, Spokane WA, Eugene OR, etc.)

Key format: `"city, state"` lowercase.

The `resolveCity(input: string)` function must handle:

- Exact match: “buffalo, new york” → match
- State abbreviation: “buffalo, ny” → expand “ny” to “new york” → match
- City only: “buffalo” → first match in map

- Aliases: “nyc” → “new york, new york”, “la” → “los angeles, california”, “sf” → “san francisco, california”, “dc” → “washington, district of columbia”, “chi” → “chicago, illinois”, “philly” → “philadelphia, pennsylvania”, “vegas” → “las vegas, nevada”, “nola” → “new orleans, louisiana”
- Case insensitive, trimmed

Filter Logic (lib/filter.ts)

```
filterDestinations(cities: City[], params: SearchParams): ScoredCity[]
```

1. Resolve home city → coords. If null, return [].
2. Calc Haversine distance (MILES) to every city.
3. Filter: distance ≤ maxDistance.
4. Mode filter: drive → only ≤ 800mi. Fly → only ≥ 200mi.
5. Score: +10 per matching vibe, +5 if current season in bestSeasons, -1 per 100mi distance (floored). Score min = 0.
6. If user selected ≥1 vibe, filter out cities with score 0.
7. Sort: score DESC, then name ASC (tiebreaker).
8. Return top 8.
9. Attach weatherHint to each via getWeatherHint().

Haversine (lib/distance.ts)

Standard Haversine formula. Earth radius = 3958.8 MILES. Returns number (miles). Zero dependencies.

Seasons (lib/seasons.ts)

getCurrentSeason(): returns ‘spring’ | ‘summer’ | ‘fall’ | ‘winter’ based on current month.

- Months 2-4 (Mar-May) = spring
- Months 5-7 (Jun-Aug) = summer
- Months 8-10 (Sep-Nov) = fall
- Months 11, 0, 1 (Dec-Feb) = winter

getWeatherHint(city: City): string

- If current season in bestSeasons → "☀️ Great time to visit!"
- Else → "Best in {firstBestSeason}"

Share Utils (lib/shareUtils.ts)

```
async function copyToClipboard(text: string): Promise<boolean>
```

- Try `navigator.clipboard.writeText(text)` first
- Fallback to creating textarea element + `document.execCommand('copy')`
- Return true on success, false on failure

Card share text format: 🧭 `Wanderlust pick: {city}, {state} ({distance} mi) — "{first sentence of blurb}"`

Dark Mode (components/DarkModeToggle.tsx)

'use client' component:

- State: `isDark: boolean`
- On mount (useEffect): check `window.matchMedia('(prefers-color-scheme: dark)').matches`, set state + toggle class
- On click: flip state, toggle `document.documentElement.classList`
- Icon: show 🌙 when light (click for dark), show ☀️ when dark (click for light)
- Position: absolute top-right of hero section
- ARIA: `aria-label="Toggle dark mode"`
- NO localStorage, NO cookies, NO next-themes

City Autocomplete (components/CityAutocomplete.tsx)

'use client' component:

- As user types, filter cityLookup keys that include the input (case-insensitive)
- Show max 8 suggestions in a dropdown below input

- Keyboard navigation: ArrowDown/ArrowUp to move, Enter to select, Escape to close
- On select, update the parent's homeCity state with the formatted city name
- Click outside closes dropdown
- Dropdown: absolute positioned, z-10, card-like styling

Design Spec

Color System (globals.css CSS custom properties)

Light mode (default — on :root): –sand: #F5E6D3; –terracotta: #C75B39; –forest: #2D5016; –cream: #FFF8F0; –charcoal: #2C2C2C; –coral: #FF6B4A; –sage: #87A878;

Dark mode (on .dark): –sand: #1A1410; –cream: #2A2420; –charcoal: #E8E0D8; –forest: #4A8A30; –sage: #6B8A60; –terracotta: #C75B39 (unchanged); –coral: #FF6B4A (unchanged);

Typography

Load via next/font/google in layout.tsx:

```
import { Fraunces, DM_Sans } from 'next/font/google';

const fraunces = Fraunces({
  subsets: ['latin'],
  variable: '--font-fraunces',
  display: 'swap',
  weight: ['400', '700', '900'],
});

const dmSans = DM_Sans({
  subsets: ['latin'],
  variable: '--font-dm-sans',
  display: 'swap',
  weight: ['400', '500', '700'],
});
```

Apply both variables to `<body>` className. Reference in Tailwind config:

```
fontFamily: {
  display: ['var(--font-fraunces)', 'serif'],
  body: ['var(--font-dm-sans)', 'sans-serif'],
}
```

Layout

- Max-width 1200px centered
- Hero: “Wanderlust” in font-display font-black text-5xl md:text-7xl. Tagline “Where should you go next?” in font-display font-normal text-xl. Dark mode toggle positioned top-right. Subtle grain texture background via CSS (inline SVG noise filter or pseudo-element, NOT an image file).
- Form: Card (bg-[var(–cream)], rounded-2xl, shadow-lg, p-6 md:p-8). Single column mobile, 2-column grid desktop (md:grid-cols-2, gap-6). Discover button spans full width below grid.
- Distance slider: Custom styled. Below slider, show: “{value} mi · {label}” where label = “Day Trip” (≤150), “Weekend Drive” (≤400), “Road Trip” (≤800), “Cross Country” (≤1500), “Fly Me Out” (>1500).
- Mode toggle: Two buttons, selected gets bg-[var(–terracotta)] text-white. 🚗 Drive | ✈️ Fly
- Trip length: Four pills. Selected = bg-[var(–forest)] text-white.
- Vibes: Flex-wrap. VibeChip unselected = border border-[var(–sage)] text-[var(–charcoal)]. Selected = bg-[var(–terracotta)] text-white, scale(1.05) transition.
- Keywords: input with opacity-60, pointer-events-none, tabIndex={-1}, with “Coming Soon ✨” badge positioned top-right of input container.
- Discover button: bg-[var(–coral)] text-white font-display font-bold text-xl py-4 rounded-lg. CSS pulse animation on idle.

Results

- Sort controls: Three text buttons above grid. Active = border-b-2 border-[var(–terracotta)] text-[var(–terracotta)].
- Grid: grid-cols-1 lg:grid-cols-2 gap-6
- Cards: bg-[var(–cream)] rounded-xl shadow-md p-6. Stagger animation via CSS @keyframes slideUp + animation-delay per card (inline style for delay only).
    - City name: font-display font-bold text-xl text-[var(–charcoal)]
    - Distance badge: inline pill bg-[var(–sage)]/20 text-[var(–sage)] rounded-full px-3 py-1 text-sm
    - Weather hint: text-xs. Green text for “Great time!”, muted for “Best in…”

- Blurb: font-body text-sm text-[var(–charcoal)] mt-2
- Vibe pills: flex-wrap gap-1.5 mt-3. Each: text-xs bg-[var(–sage)]/10 rounded-full px-2 py-0.5
- Best seasons: text-xs text-[var(–sage)] mt-2
- Share button: absolute top-4 right-4, small icon button. On click → copy to clipboard, show "Copied!" tooltip for 2s.

Empty / Error States

- No search yet: centered text "Tell us what you're looking for and we'll find your next adventure 🧭", font-display italic
- No results: "No destinations matched — try widening your distance or picking different vibes!"
- City not found: red text below home city input: "Couldn't find that city — try 'City, State' format"

Footer

Low opacity (opacity-40), small text (text-xs), centered at bottom, py-8. Line 1: "100% client-side. No data stored. No tracking. Your info never leaves your browser." Line 2: "@ReelDad on X" — anchor tag linking to https://x.com/ReelDad, same subtle styling. No other personal info. No version numbers. No company names. No project names.

CSS Keyframe Animations (in globals.css)

```
@keyframes fadeIn {
  from { opacity: 0; }
  to { opacity: 1; }
}

@keyframes slideUp {
  from { opacity: 0; transform: translateY(20px); }
  to { opacity: 1; transform: translateY(0); }
}

@keyframes pulse {
  0%, 100% { opacity: 1; }
  50% { opacity: 0.85; }
}
```

Accessibility

- All interactive elements: visible focus ring (outline-2 outline-[var(–coral)] outline-offset-2)
- VibeChips: role="checkbox", aria-checked={selected}
- Mode toggle: role="radiogroup" with role="radio" per button, aria-checked
- Slider: aria-label, aria-valuetext (e.g. "500 miles, Road Trip")
- Dark mode toggle: aria-label="Toggle dark mode"
- Skip to content link (hidden, visible on focus)

API Stub

/app/api/suggest/route.ts:

```
import { NextResponse } from 'next/server';

export async function POST() {
  return NextResponse.json(
    { message: 'AI suggestions coming soon — for now, enjoy our curated picks!' }
    { status: 501 }
  );
}
```

Build Checklist (Self-Audit Before Finishing)

After building all files, verify:

- All 120 cities present with real coords (spot-check 5 random cities against Google Maps)
- Haversine returns miles (test: NYC to LA should be ~2445mi)
- No `any` types anywhere
- No localStorage / sessionStorage / cookies / analytics
- Dark mode works: toggle adds/removes 'dark' class, all colors switch
- CityAutocomplete handles "nyc", "NYC", "new york", "New York, NY"
- Keywords input is visible but not interactive (pointer-events-none)

- Footer has exactly 2 lines as specified, no other personal info
- All ‘use client’ directives present on client components
- API stub returns 501
- No console.log statements
- Share button copies formatted text to clipboard
- Responsive: mobile single column, desktop 2-column
- Sort controls work (best match, closest, furthest)
- Empty state shows when no search performed
- Error state shows when city not found

Build every single file now, complete and production-ready. Start with types and data files, then lib utilities, then components, then app files. State what you’re building before each file.

—PROMPT END—

Post-Build Notes

- **Domain**: `wanderlust.vercel.app` is likely taken. Try: `getwanderlust.vercel.app`, `trywanderlust.vercel.app`, `wanderlust-trips.vercel.app`, or `where-next.vercel.app`
- **Future AI integration**: Swap `/api/suggest` stub with real Claude API call. Frontend already captures keywords. Toggle between client-side filter and AI mode. AI mode sends SearchParams + keywords, Claude does web search for events/deals/weather, returns enriched suggestions.
- **Performance**: 120 cities + Haversine on every search = trivial computation. No optimization needed for MVP. If dataset grows to 1000+, consider pre-computing distance matrices or using a spatial index.